Database Systems Core

Types, Purpose, Scaling & Security

Amjad Hossain

28th June, 2026

Why Database Choice Matters

The database becomes the long-term memory of the system.

- A wrong database choice creates performance, cost, and maintenance pain.
- Most systems fail not because the database is bad.
- They fail because the database was used for the wrong purpose.

But… but… everything is just data!!

Customer orders are data.

- Product searches are data.
- User sessions are data.
- Audit logs are data.
- Analytics reports are data.
- But they do not behave the same way.

Common Problems With Wrong Database Choice

- Slow queries under real production load
- Complex joins inside a database that was not designed for joins
- No clear transaction boundary
- Expensive scaling path
- Hard backup, restore, and disaster recovery
- Security and compliance gaps
- Too much operational responsibility for the team

The Database Is Not Just Storage

A database usually gives us:

- Data model
- Query model
- Consistency rules
- Transactions
- Indexing
- Security boundary
- Backup and recovery capability

First Question

What shape does the data have?

- Rows and columns?
- Nested JSON documents?
- Key and value?
- Relationships?
- Events over time?
- Full-text search documents?

Second Question

How will the application access the data?

- Transactional writes?
- Fast reads?
- Search?
- Reporting?
- Graph traversal?
- Cache lookup?
- Time-window aggregation?

1

Different Database Types

Purpose first. Technology second.

Database
Architecture
& Operations

Different Database Types

Type	Simple Purpose
Relational / SQL	Structured data, ACID transactions, strong consistency
Document	Flexible JSON-like data and evolving schemas
Key-Value	Very fast lookup by key
Wide-Column	Huge scale, high write throughput, sparse rows
Search Engine	Full-text search, ranking, filtering, log search
Graph	Relationship-heavy data and traversals
Time-Series	Metrics, sensor data, events over time
Columnar / OLAP	Analytics and reporting over large datasets
Vector	Similarity search for embeddings and semantic retrieval

Relational / SQL Database

- Data is stored in tables.
- Relationships are handled through primary keys and foreign keys.
- SQL is used for querying.
- Transactions protect correctness.

Common examples: PostgreSQL, MySQL, SQL Server, Oracle Database.

Use Relational DB When

- You need strong consistency and ACID transactions
- Data has clear structure and relationships
- Business rules depend on correctness
- You need joins, constraints, reports, and mature tooling
- Finance, orders, inventory, HR, billing, academic records

Document Database

Data is stored as documents, usually JSON-like structures.

- A document can contain nested data.
- Schema can evolve faster than relational schema.
- Common examples: MongoDB, Couchbase, Azure Cosmos DB document model.

Use Document DB When

- Data naturally comes as nested documents
- Schema changes frequently
- Reads usually fetch the whole document
- You want to avoid complex joins
- Product catalog, user profile, content management, dynamic forms

Key-Value Database / Cache

The data access pattern is very simple:

- give me value for this key
- Common examples: Redis, Memcached, DynamoDB key-value pattern.

Use Key-Value Store When

- You need extremely fast lookup
- The application already knows the key
- Data can be temporary or reconstructed
- Session storage, cache, rate limiting, OTP, feature flags

Wide-Column Database

Designed for very large scale and high write throughput.

- Rows can have many columns and sparse data.
- The data model is usually query-driven.
- Common examples: Cassandra, HBase, ScyllaDB.

Use Wide-Column DB When

- Write volume is very high
- Data is distributed across many machines
- Query patterns are known in advance
- You can design tables around access patterns
- IoT events, telecom records, large-scale activity feeds

Search Engine Database

Optimized for searching text and filtering documents.

- Supports ranking, analyzers, tokenization, and inverted indexes.
- Common examples: Elasticsearch, OpenSearch, Solr.

Use Search Engine When

- Users need full-text search
- Ranking and relevance matter
- Filtering, faceting, and log exploration are important
- Search query is more important than transaction correctness
- Product search, log search, document search, audit search

Graph Database

Optimized for relationships between entities.

- The relationship is often as important as the entity.
- Common examples: Neo4j, Amazon Neptune, JanusGraph.

Use Graph DB When

- You need to traverse many relationships quickly
- Relationships change and grow frequently
- A relational join model becomes too complex
- Fraud detection, recommendation, social network, dependency mapping

Time-Series Database

Optimized for data over time.

- The query usually asks: what happened during this time window?
- Common examples: TimescaleDB, InfluxDB, Prometheus-style storage.

Use Time-Series DB When

- Data arrives continuously
- Time is the main dimension
- You need retention, downsampling, and aggregation
- Monitoring metrics, sensors, financial ticks, application telemetry

Columnar / OLAP Database

Optimized for analytics, reporting, and aggregations.

- Reads many rows but only selected columns.
- Common examples: Snowflake, BigQuery, Redshift, ClickHouse, Synapse.

Use OLAP / Warehouse When

- Business users need reports and dashboards
- Queries scan large historical datasets
- Data comes from many source systems
- You should not overload the production OLTP database

Vector Database

Stores vectors/embeddings for similarity search.

- It answers: which items are semantically close to this item?
- Common examples: pgvector, Pinecone, Milvus, Weaviate, Qdrant.

Use Vector Search When

- You need semantic search instead of exact keyword search
- You need recommendation or similarity matching
- You are building RAG, knowledge search, image similarity, duplicate detection
- Usually not the primary system-of-record database

2

When To Use What Database

Start from workload, not from brand name.

Database
Architecture
& Operations

When To Use What Database

Need	Good Fit
Money movement, orders, inventory, billing	Relational SQL database
Flexible nested product/profile/content document	Document database
Fast lookup, cache, session, token	Key-value store
Full-text search and ranking	Search engine
Metrics/events over time	Time-series database
Relationship traversal	Graph database
Dashboards and historical analytics	OLAP warehouse
Semantic similarity	Vector database

Simple Decision Matrix

Question	If YES, think about
Do I need multi-row ACID transaction?	Relational DB
Is one entity a large nested document?	Document DB
Is access always by key?	Key-value store
Do users search free text?	Search engine
Is time the main filter?	Time-series DB
Are relationships the main problem?	Graph DB
Is the workload mostly reporting?	OLAP / warehouse
Is the query about semantic similarity?	Vector DB

A Very Common Real System

- Core transactional data: PostgreSQL / SQL Server / Oracle
- Cache and sessions: Redis
- Search: Elasticsearch / OpenSearch
- Analytics: Data warehouse
- Files and images: Object storage

- Core transactional data: PostgreSQL / SQL Server / Oracle
- Cache and sessions: Redis
- Search: Elasticsearch / OpenSearch
- Analytics: Data warehouse
- Files and images: Object storage

One system may need multiple data stores.

Example: eCommerce Data Stores

```
eCommerce Platform
├── Orders, Payments, Inventory        → Relational DB
├── Product Search                     → Search Engine
├── Shopping Cart / Session            → Key-Value Store
├── Product Images                     → Object Storage
├── Clickstream / User Activity        → Event Log
└── Sales Dashboard                    → OLAP Warehouse
```

Architecture View

Architecture decision: one database type should not be forced to solve every data problem.

Do Not Start With Polyglot Complexity

For most business applications:

- Start with one strong relational database.
- Add cache only when there is a real read pressure.
- Add search only when database LIKE queries are not enough.
- Add warehouse only when reporting hurts production.
- Add specialized databases only when the use case is clear.

3

Enterprise Grade DB vs Managed DB

Oracle, SQL Server, managed cloud, and self-managed choices.

Database Architecture & Operations

What Is an Enterprise Grade Database?

- A database platform designed for mission-critical workloads.
- Strong transaction support.
- High availability and disaster recovery options.
- Advanced security, auditing, partitioning, backup tooling.
- Vendor support, ecosystem, and compliance-friendly operations.

When We Need Oracle Database

- Core banking, telecom, ERP, government, large enterprise systems
- Very strong vendor ecosystem and mature enterprise tooling
- Complex stored procedures, partitioning, replication, HA/DR requirements
- Existing Oracle skills, licenses, applications, or compliance expectations
- Workloads where downtime and data loss are extremely expensive

When We Need Microsoft SQL Server

- .NET-heavy enterprise applications
- Microsoft ecosystem: Windows Server, Active Directory, Power BI, SSIS/SSRS
- Strong T-SQL, reporting, integration, and enterprise administration tooling
- High availability with Always On Availability Groups
- Organizations already standardized on Microsoft stack

When Enterprise DB May Be Too Much

- Small CRUD application
- Low budget and no licensing plan
- No DBA or operational skill in the team
- No strict compliance or enterprise support requirement
- The workload can be served well by PostgreSQL/MySQL/managed service

What Is Managed Database Service?

The cloud provider runs the database platform for you.

- You still own schema, queries, indexes, access, cost, and data quality.
- Provider usually manages provisioning, patching, backups, monitoring hooks, HA options, and recovery primitives.

Managed Database Examples

Provider / Platform	Examples
AWS	RDS, Aurora, DynamoDB, ElastiCache, Redshift
Azure	Azure SQL Database, Managed Instance, Cosmos DB, Cache for Redis, Synapse
Google Cloud	Cloud SQL, AlloyDB, Spanner, Bigtable, BigQuery, Memorystore
Oracle Cloud	Autonomous Database, Base Database Service, Exadata Database Service
MongoDB	MongoDB Atlas
Redis	Redis Cloud / Enterprise

Use Managed DB Service When

- You want faster production readiness
- You do not want to manage patching, backup infrastructure, and failover plumbing
- Team is small or focused on product delivery
- Cloud is already accepted by the organization
- Elastic scaling and managed monitoring are important

Be Careful With Managed DB When

- Data residency or regulatory rules are strict
- Vendor lock-in risk is unacceptable
- Network latency to cloud is a problem
- You need deep OS-level or storage-level control
- Cost becomes unpredictable under heavy workload

4 In-House / Self-Managed Database

More control. More responsibility.

Database Architecture & Operations

What Is In-House Database System?

You run the database yourself.

- It can be on physical servers, on-prem VMs, private cloud, or cloud VMs.
- The organization owns installation, upgrade, backup, failover, monitoring, security, and disaster recovery.

Use In-House Database When

- Regulation requires local control of data
- Existing datacenter and DBA team already exist
- Special hardware, network, or storage setup is required
- Cloud connectivity is unreliable or not allowed
- At very large scale, self-managed cost may be lower if the team is mature

In-House Database Responsibilities

- Provisioning and capacity planning
- OS and database patching
- Backup, restore, and disaster recovery testing
- Replication, failover, clustering
- Monitoring, alerting, log retention
- Security hardening and audit readiness
- 24/7 incident response

In-House Database Architecture Example

```
Company Datacenter
├── App Server 1
├── App Server 2
├── Database Primary
├── Database Standby / Replica
├── Backup Storage
├── Monitoring Server
├── Firewall / Network Segmentation
└── Offsite DR Location
```

Architecture View

A Practical Rule

If the team cannot confidently answer these questions:

- How do we restore last night’s backup?
- How do we fail over if the primary server dies?
- Who receives alerts at 3 AM?
- How do we rotate credentials?
- Then managed service is usually safer.

5

Different Approaches To Scaling Database

Scaling is not only adding bigger machines.

Database Architecture & Operations

Before Scaling

Measure first.

- Slow query log
- Execution plan
- Missing indexes
- Wrong data model
- N+1 query problem
- Connection pool exhaustion
- Bad transaction design

Database Scaling Approaches

Approach	Simple Meaning
Query tuning	Fix SQL, indexes, execution plans, transactions
Vertical scaling	Bigger CPU, RAM, disk IOPS
Read replicas	Offload read traffic
Caching	Avoid repeated database reads
Partitioning	Split large table logically inside DB
Sharding	Split data across multiple database nodes
CQRS / read model	Separate write model from read-optimized views
Archiving	Move old data away from hot operational tables

Approach 1: Query and Index Tuning

Usually the first and cheapest scaling step.

- Add the right indexes.
- Remove unused indexes that slow writes.
- Fix bad joins and N+1 queries.
- Keep transactions short.
- Review execution plans before buying bigger hardware.

Approach 2: Vertical Scaling

Increase CPU, RAM, storage, or IOPS of the database server.

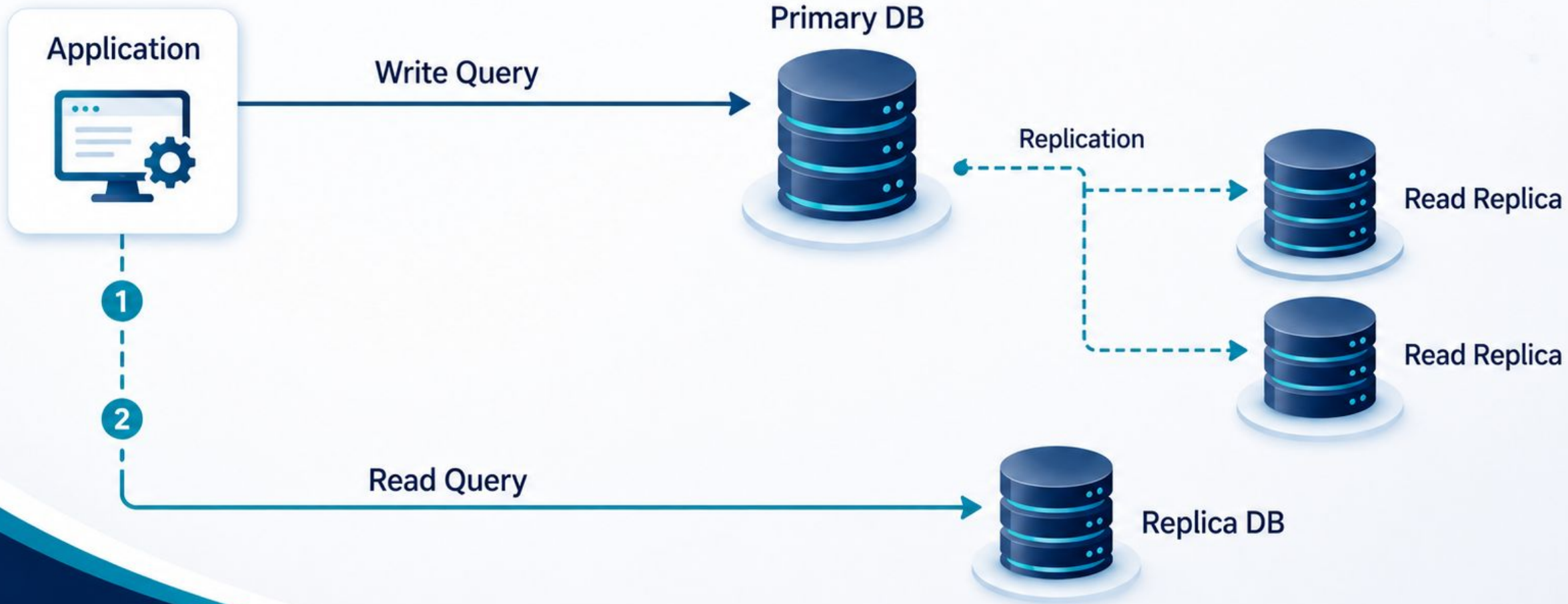
- Simple to operate.
- Good for early growth.
- But there is always a maximum machine size.
- Cost can rise sharply at the high end.

Approach 3: Read Replicas

Primary database handles writes.

- Replica databases handle reads.
- Good for read-heavy systems.
- But replication lag must be understood.
- Do not read stale data when business correctness needs fresh data.

Read Replica Pattern



Approach 4: Caching

Cache frequently read data.

- Use Redis or in-memory cache.
- Reduce load on the primary database.
- Main problem: cache invalidation.
- Never let cache become the hidden source of truth.

Approach 5: Partitioning

Split a large table into smaller logical parts.

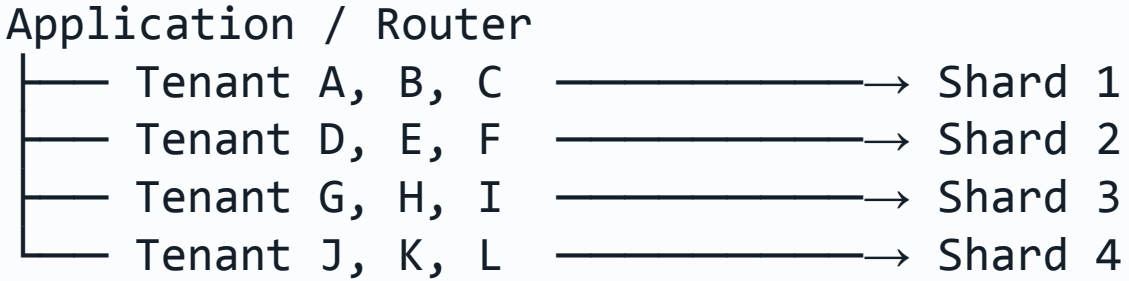
- Partition by date, tenant, region, or hash.
- Improves maintenance and query pruning.
- Useful for large historical tables.
- Still usually one database cluster.

Approach 6: Sharding

Split data across multiple database nodes.

- Shard by tenant, customer, region, or hash.
- Can scale writes and storage horizontally.
- But cross-shard query and transaction become harder.
- Shard only when simpler approaches are not enough.

Sharding Pattern



Architecture View

The shard key is one of the most important long-term architecture decisions.

Approach 7: CQRS / Read Models

Write model protects business rules.

- Read model is optimized for screen/query/report needs.
- Useful when reads and writes have very different shapes.
- But it introduces eventual consistency and synchronization complexity.

Approach 8: Archiving and Retention

- Old data should not always stay in hot operational tables.
- Move historical data to archive tables, warehouse, or object storage.
- Keep the primary database smaller and faster.
- Define retention from business and compliance requirements.

Approach 9: Connection Pooling

- Database connections are expensive.
- Application servers should not open unlimited connections.
- Use connection pooling correctly.
- Set max pool size based on database capacity.
- Connection storm can take down a healthy database.

Approach 10: Multi-Region Database

Used when availability, latency, or disaster recovery requires multiple regions.

- Active-passive is simpler.
- Active-active is harder.
- Data consistency, conflict resolution, and cost become major design concerns.

6 Database Security Best Practices

Security is not one feature. It is layered control.

Database Architecture & Operations

Database Security Layers

- Network security
- Identity and access control
- Encryption
- Secret management
- Audit logging and monitoring
- Backup and recovery protection
- Patching and hardening
- Application-level protection

Network Security

- Database should not be open to the public internet.
- Place database inside private network/subnet.
- Allow only required application servers or bastion hosts.
- Use firewall/security group rules strictly.
- Separate production, staging, and development networks.

Authentication and Authorization

- Use least privilege.
  - Separate app user, migration user, read-only user, and DBA/admin user.
- Do not use root/sysadmin user from application code.
- Remove default accounts and unused users.
- Review permissions regularly.

Encryption

- Encrypt data at rest.
- Encrypt data in transit using TLS.
- Protect backup files with encryption.
- Manage keys carefully.
- Remember: encryption does not fix bad access control.

Secrets Management

- Do not store database passwords in source code.
- Use secret manager or secure vault.
- Rotate credentials.
- Use different credentials per application/environment.
- Audit who can read or change secrets.

Audit Logging and Monitoring

- Log security-relevant events.
- Monitor failed login attempts.
- Detect unusual query volume or data export pattern.
- Alert on replication lag, disk pressure, backup failure, and high error rates.
- Keep logs tamper-resistant where possible.

Backup Security

- Backups are production data.
- Encrypt backups.
- Restrict backup access.
- Store backups in a separate failure domain.
- Test restore regularly.
- A backup that was never restored is only a hope.

Patching and Hardening

- Patch database engine and operating system.
- Disable unused features and extensions.
- Run database service with low privilege account where applicable.
- Remove default databases/accounts that are not needed.
- Baseline configuration and detect drift.

Application-Level Database Security

- Use parameterized queries.
- Do not concatenate user input into SQL.
- Validate input at application boundary.
- Avoid over-fetching sensitive columns.
- Mask or tokenize sensitive data when possible.
- Do not expose internal database errors to users.

Production Database Checklist

- Private network only
- Least privilege users
- Encrypted at rest and in transit
- Automated backups and tested restore
- Monitoring and alerting
- Patch plan
- Audit logging
- Documented failover and incident runbook

7

Practical Architecture Guidance

A simple path for real-world systems.

Database Architecture & Operations

Recommended Starting Point

For most business systems:

- Start with a relational database.
- Keep schema clean and normalized enough.
- Use indexes carefully.
- Add Redis only for proven cache/session needs.
- Add search engine only for real search requirements.
- Add warehouse only when analytics grows.

Small Product / MVP

- Managed PostgreSQL / MySQL / SQL Server is usually enough.
- One primary database.
- Automated backup enabled.
- Basic monitoring and alerts.
- Use migrations or controlled SQL scripts.
- Avoid unnecessary specialized databases.

Growing Business System

- Add read replicas for read pressure.
- Add Redis for cache/session/rate limit.
- Add search engine for product/document search.
- Separate reporting workload from OLTP.
- Introduce stricter security and audit process.
- Define RPO and RTO clearly.

Enterprise / Mission-Critical System

- Formal HA/DR architecture
- Failover drill and restore drill
- Data classification and access review
- Audit trail and compliance reporting
- Capacity planning and performance testing
- Dedicated DBA/platform ownership
- Clear vendor support path

Final Rule of Thumb

Choose the database by responsibility:

- System of record → relational / enterprise DB
- Fast temporary lookup → cache / key-value
- Search experience → search engine
- Historical analytics → warehouse / OLAP
- High-volume time events → time-series / event pipeline
- Relationship traversal → graph database

What Good Database Architecture Looks Like

- Correctness first
- Performance based on measurement
- Security by default
- Backup and restore proven by testing
- Scaling path known before emergency
- Operational responsibility clearly owned
- No database chosen only because it is popular

References Used

Area	Reference
Managed relational DB	AWS RDS documentation; Azure SQL Database overview
SQL Server enterprise/HA	Microsoft SQL Server editions; Always On Availability Groups docs
Database security	OWASP Database Security Cheat Sheet; OWASP Secrets Management Cheat Sheet
Scaling concepts	PostgreSQL HA/load balancing docs; MongoDB sharding docs
Cloud examples	Official cloud provider documentation and product pages

Thank you